

Final PPSE Project Report

Amoura Zahir Sami

Contents

1	First Steps of Optimization	2
1.1	Quantization	2
1.2	Neon Function	3
1.2.1	Soft Decoder	4
1.2.2	Hard Decoder	4
2	Mini Project Grounds	4
2.1	Modulator (Task 1)	4
2.2	Demodulator (Task 2)	5
2.3	Monitor (Task 3)	5
2.4	Bit Packing (Task 4)	5
2.5	Encoder (Bonus Task)	6
2.6	MT19937 PRNG Optimization	6
2.6.1	High-Level Algorithm	7
2.6.2	Low-Level Algorithm Parameters	7
2.6.3	Mersenne Twister Conclusion	9
2.7	Channel Optimization	9
3	Throughput Performance Metrics Comparison	9
3.1	Simulation 1 Throughput Comparison (at 15 dB SNR)	10
3.2	Simulation 2 Throughput Comparison (at 12 dB SNR)	10
3.3	Simulation 3 Throughput Comparison (at 12 dB SNR)	11
3.4	Simulation 4 Throughput Comparison (at 12 dB SNR)	11
3.5	Simulation 5 Throughput Comparison (at 12 dB SNR)	11
3.6	Summary & Key Findings	12
3.6.1	Simulation Throughput Performance Summary Table	12
3.6.2	Simulation FER/BER graphs	12

Initial Results and graph

Bit Error Rate (BER) and Frame Error Rate (FER) graphs at the end of the simulation chain. The simulation stopped at 15 dB for the hard decoder and 12 dB for the soft decoder.

The soft decoder proves to be more efficient and demonstrates better performance rates than the hard decoder.

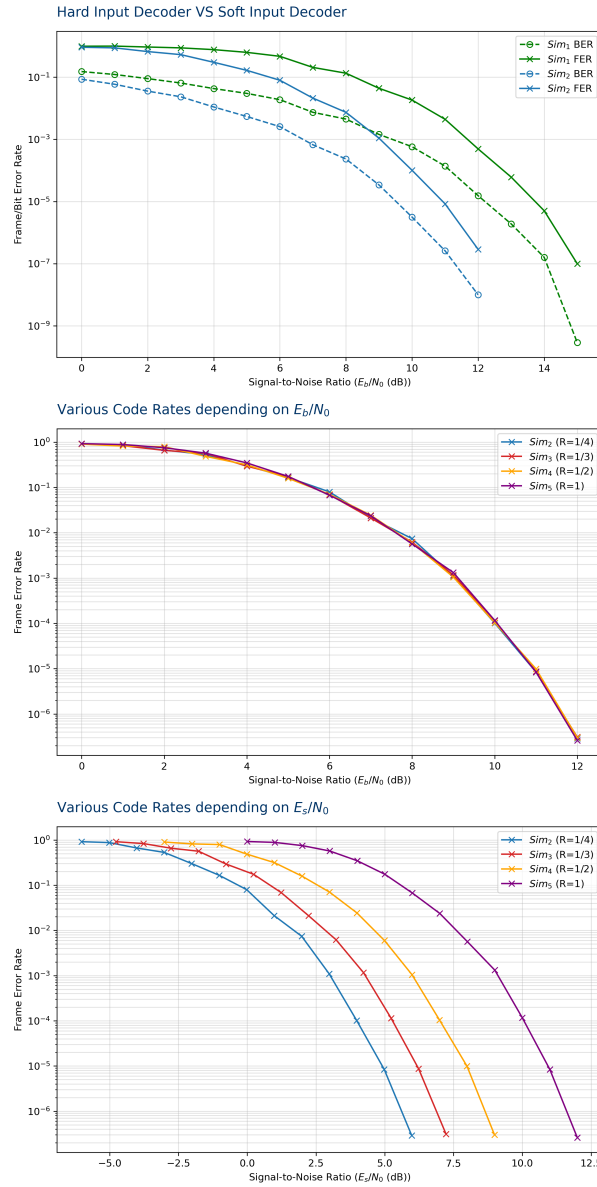


Figure 1: Lab 3 graphs

1 First Steps of Optimization

1.1 Quantization

The primary objective here was to achieve a better simulation throughput. The strategy involved accelerating the decoder block by implementing a quantizer that transforms values from `float` to `int8`, followed by utilizing a special decoding function compatible with the `int8` format.

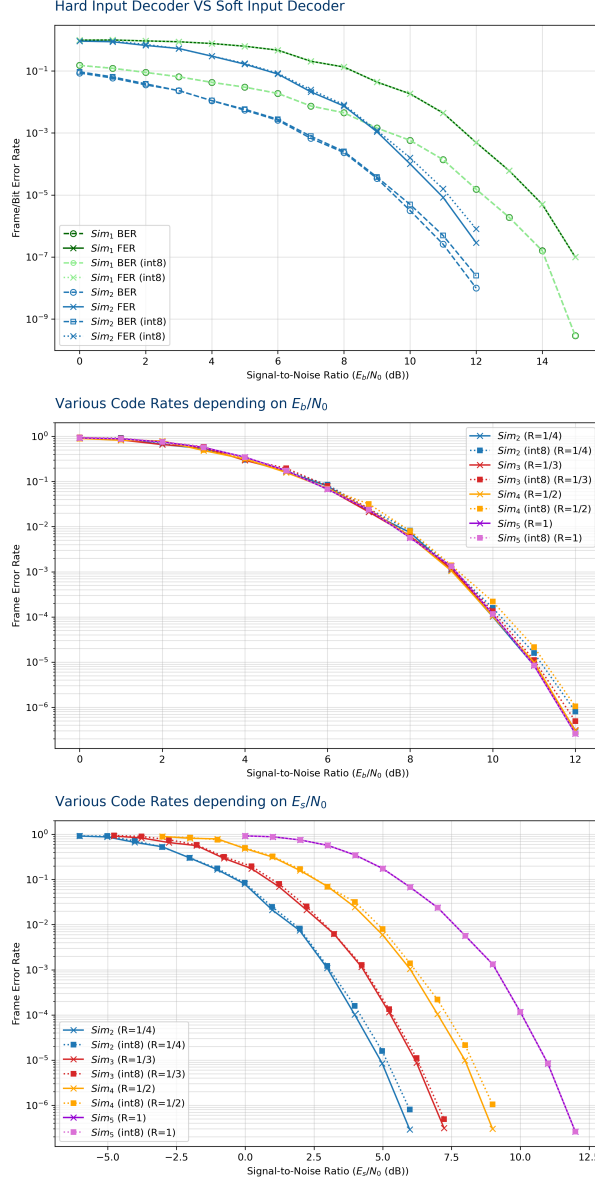


Figure 2: Lab 4 graphs

Comparing the BER/FER curves with and without `int8`, we observe a degradation in BER/FER performance. This is explicitly due to the loss of precision caused by quantization.

Conversely, for the hard decoder, no performance loss was noticed since it only requires the bit sign; thus, quantization does not affect its BER/FER curve.

Note: For $R = 1$, the soft decoder behaves exactly like a hard decoder, yielding identical results with or without `int8`.

1.2 Neon Function

Even though the quantizer block was introduced, the global simulation throughput did not see a significant baseline improvement. The next strategy was to leverage the Nano's SIMD functions compatible with the `int8` format.

For simulation testing, the parameters were set to $K = 32$ and $N = 8192$ (the high number of repetitions will be discussed later). The primary focus remains on the Decoder's individual

throughput. As a baseline, the decoder compatible with the `float` format was used (results are available in the `../simulations/compare/` directory).

1.2.1 Soft Decoder

- **Baseline:** An average throughput of 0.7 Mbps for the decoder, with the global simulation throughput standing at 0.03 Mbps.
- **With Quantizer Block:** Yielded practically identical throughput results with a slightly degraded simulation throughput.
- **With SIMD Functions:** Decoder throughput expanded to 5.5 Mbps. This marks a massive block improvement, though global simulation throughput stayed relatively the same.

A visible degradation in the FER/BER curves was noticed due to two distinct factors:

1. The critical loss of numerical precision stemming from the quantizer block.
2. Saturation inside the decoder when utilizing NEON functions. The task in Lab 4 strictly constrained the instructions allowed. Consequently, we could only use `int8` for the soft decoder. When calculating the summation over 256 repetitions, there is a very high likelihood that the `int8` variable saturates (since `int8` holds values up to 127). This saturation directly caused the deterioration of the FER and BER curves.

1.2.2 Hard Decoder

- **Baseline:** An average throughput of 0.2 Mbps for the decoder, with a global simulation throughput of 0.029 Mbps.
- **With Quantizer Block:** Practically the same throughput results as discussed earlier, with a slightly lower simulation throughput of 0.025 Mbps.
- **With SIMD Functions:** Decoder throughput increased to 1.3 Mbps. Though global simulation throughput did not increase.

2 Mini Project Grounds

Before commencing, the decoder code was modified to remove dynamic memory allocation (`malloc`) and buffer copies (`memcpy`). This minor adjustment bumped the decoder's throughput (only for the hard decoder, as the soft decoder did not contain `malloc`). Moving forward, the focus will reside on **Axe 2** of the miniproject.

2.1 Modulator (Task 1)

To begin with, the mapping of the X_N result to an `int32` array was discarded to avoid over-complicating processing loops. Instead, X_N is structured as an `int8` array, offering a clean 1:1 correspondence with the input from the encoder.

The channel block still expects `int32` inputs (will change it later). To isolate and test the throughput of the Modulator block, the `int8` data is converted to `int32` just outside the block.

- **Baseline ($K = 32, N = 128$):**
 - Average throughput before vectorization: 1.4 Gbps.
 - Average throughput after vectorization: 2 Gbps.

A clear gain that scales with larger values of N :

- **Baseline** ($K = 32, N = 8192$):
 - Average throughput before vectorization: 3 Gbps.
 - Average throughput after vectorization: 12 Gbps.

To activate this optimized module, run the simulation with the `--neonmod` flag.

Function Signature:

```
void modem_bpsk_modulate_neon(const uint8_t *C_N, int8_t *X_N, size_t N
    );
```

Additionally, the channel was adapted to the new input format by creating a secondary function taking `int*` pointers rather than native `int32` scalar logic.

2.2 Demodulator (Task 2)

Vectorization was implemented utilizing the `float32x4_t` NEON vector layout to keep implementation logic simple for the time being. No major modifications or jumps in throughput were recorded during isolated executions.

To activate this module, utilize the execution flag `--neondemod`.

Function Signature:

```
void modem_BPSK_demodulate_neon(const float *Y_N, float *L_N, size_t N,
    float sigma);
```

2.3 Monitor (Task 3)

To vectorize the error-checking routine, a simple parallel approach was adopted: executing an bitwise XOR operation between the output of the decoder and the original message vector, followed by accumulating all set bits across the parallel result lanes to determine total errors. This optimization effectively doubles the throughput of the monitor block.

To activate this module, utilize the execution flag `--neonmon`.

Function Signature:

```
void monitor_check_errors_neon(const uint8_t *U_K, const uint8_t *V_K,
    size_t K, uint64_t *n_bit_errors, uint64_t *n_frame_errors);
```

2.4 Bit Packing (Task 4)

This technique is utilized to store 8 individual bits inside a single byte, rather than inflating memory footprints by storing only 1 bit per byte.

We apply this optimization starting directly at the source block: instead of streaming singular binary 0 or 1 integers, the source can generate random values spanning 0 to 255 (representing 8 random bits of generated information). However, for this pipeline to remain compatible downstream, the stream must be unpacked immediately prior to hitting the modulator block (as the modulator relies on separated binary layouts).

To test the raw performance of this packed source generator, use the configuration flag `--packed`. Operating this module yields a marginally lower nominal throughput metric, but it generates 8 times less volume ($K/8$ bytes instead of K bytes).

Function Signature:

```
void source_generate_packed(uint8_t *U_K_packed, size_t K);
```

Note: Measured FER and simulation trajectories will fluctuate slightly due to the modification of the PRNG bit extraction sequence, but variance remains negligible.

For packing and unpacking data segments, the helper definitions are:

```
void pack_bits(const uint8_t *unpacked, uint8_t *packed, size_t
    num_unpacked);
void unpack_bits(const uint8_t *packed, uint8_t *unpacked, size_t
    num_packed);
```

For the isolated unpacking routine, throughput hovers around 370 Mbps on average when evaluating configurations at $K = 32$. Vectorizing this specific node may prove overly complex relative to its practical speed return, but might check it out if time allows it. (pack bit was unused during the project).

2.5 Encoder (Bonus Task)

The repetition encoder is inherently one of the slower execution blocks in the standard loop (excluding the channel), it scales effectively with SIMD vectorization since it consists almost entirely of structured load and store operations.

We can pull a complete vector lane using `vld1q_u8`, then store it back using calculated offsets directly inside the destination C_N sequence, repeating this iteration sequence uniformly to fulfill the repetition target. This requires K to be a multiple of 16 since vector load registers evaluate elements 16 items at a time. The targeted memory displacement is calculated by multiplying the active repetition count with the input dimension.

Pseudo-code Architecture:

```
for rep in reps {
    offset = rep * K;
    i = 0;
    for (; i < K; i += 16) {
        vector_reg = load(UK[i]);
        store(vector_reg into CN[offset + i]);
    }
}
```

This implementation choice boosts the independent throughput of our encoder block by roughly 8 times due to the massive reduction in individual load/store instructions and processing 16 items concurrently.

Function Signature:

```
void codec_repetition_encode_neon(uint8_t *U_k, uint8_t *C_N, size_t K,
    size_t n_reps);
```

2.6 MT19937 PRNG Optimization

The communication channel represents the bottleneck within the simulation loop due to its intense mathematical calculation requirements. This phase explicitly isolates and optimizes the classic `rand()` loop, replacing it with a custom vectorized pseudo-random number generator implementing the Mersenne Twister (MT19937) algorithm.

I will link below an amazing video i stumbled upon doing research as well as wikipedia documents going through more details:

- Mersenne Twister Engine Video Analysis
- Wikipedia Documentation Source

2.6.1 High-Level Algorithm

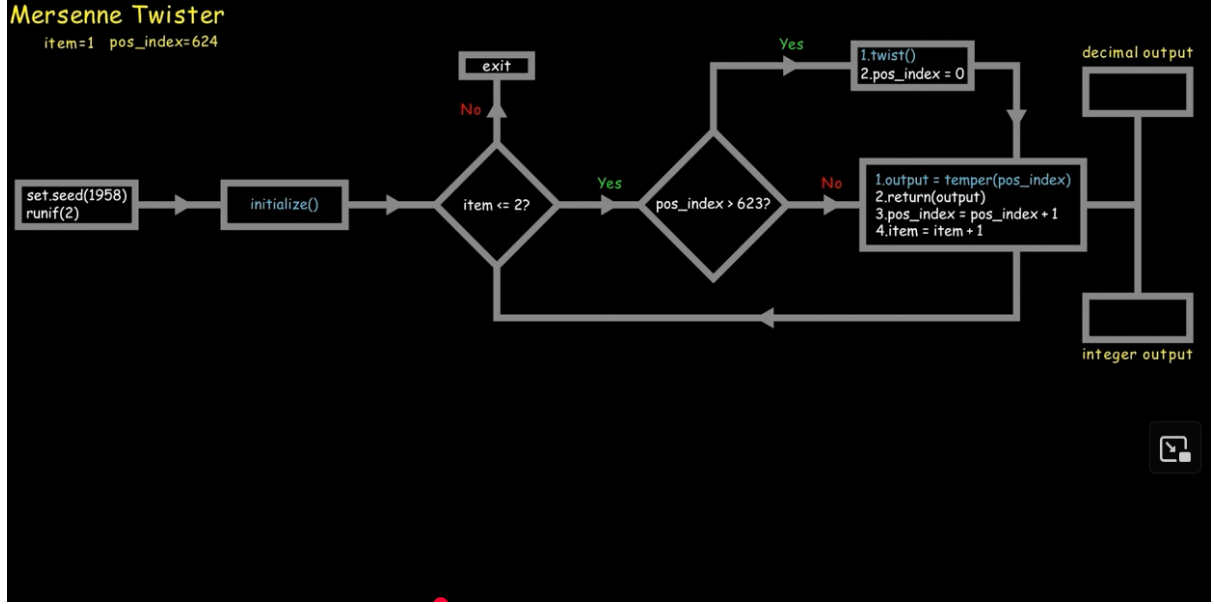


Figure 3: high-level Mersenne Twister diagram

Every random sequence initialization begins from an input seed value. The initial phase scales this distinct seed outward to populate a fixed State array tracking 624 independent 32-bit internal states.

Then we do a twist, to create the twisted MT matrix we use 3 elements from the original array we created from growing the seed (i , $i+1$ and $i+397$). Once the twisted MT completed we derive from each element of this array a random number.

An internal index tracker increments with each random draw. Upon hitting index 623, the array is re-twisted and the tracker resets back to 0.

Summary of Processing Pipeline:

- **Initialization:** Expand the base seed across the 624-element state matrix. Internal pointer array position initializes at index 624.
- **Twisting:** Execute vector twist operations across the states and drop pointer positions back down to index 0.
- **Generation:** Inline bit-tempering extracts a finalized pseudo-random integer using 3 structural elements from the twisted matrix, incrementing the pointer index sequentially.
- **Looping:** When reaching the array bounds (index pointer = 624), re-twist the matrix array to obtain a completely new state sequence.

The standard MT19937 formulation guarantees a maximum sequence recurrence period of $2^{19937} - 1$.

2.6.2 Low-Level Algorithm Parameters

The computational parameters are globally fixed according to standard specification constraints:

- $(w, n, m, r) = (32, 624, 397, 31)$
- $a = 0x9908B0DF$

- $(u, d) = (11, 0xFFFFFFFF)$
- $(s, b) = (7, 0x9D2C5680)$
- $(t, c) = (15, 0xEFC60000)$
- $l = 18$

Initialization: To expand the input seed and populating the MT array structure, the sequence evaluation follows:

$$x_i = f \times (x_{i-1} \oplus (x_{i-1} \gg (w - 2))) + i \quad (1)$$

Where f for standard MT19937 configurations equates precisely to 1812433253.

Note: This stage is strictly sequential and cannot be vectorized. Because it executes once at the program startup, no impact on performance.

Twisting Process: The following sections elaborate specifically on the 4-lane parallelized SIMD implementation.

```
def twist():
    for i in 0:623:
        x = (MT[i] & upper) + (MT[(i+1) % n] & lower)
        xA = x >> 1
        if (x % 2) != 0:
            xA = xA ⊕ a
        Twisted_MT[i] = MT[(i + m) % n] ⊕ xA
```

Figure 4: Twisting algorithm

Tempering Step: This specific operations block extracts the final random configuration by masking the state variable through bitwise operations using the constants declared above:

```
y ^= (y >> u);
y ^= (y << s) & b;
y ^= (y << t) & c;
y ^= (y >> l);
```

Vectorization Strategy: The main optimization goal was to vectorize both the *Twisting* and *Tempering* pipelines (with initialization remaining strictly scalar).

- **Twisting Vectorization:** The primary challenge involved handling tracking indexes. Generating the twisted states requires retrieving 3 entries concurrently (i , $i + 1$, and $i + m$), causing a circular index wrap-around to occur around index 227. To solve this, processing loops were split into three independent data zones:

1. Indexes spanning from 0 to 226, fetching targets i , $i + 1$, and $i + m$.
2. Indexes spanning from 227 to 622, fetching targets i , $i + 1$, and $i + m - n$.
3. Edge boundaries isolating arrays cells 622 and 623, which are computed sequentially.

The logic to parallelize these sequences is straightforward since operations are limited to bitwise logic, which maps directly to SIMD instructions. The tempering block follows the same approach. Both calculations utilize `uint32x4_t` NEON layouts.

2.6.3 Mersenne Twister Conclusion

Vectorizing the pseudo-random number generator significantly improved global simulation throughput, yielding a speedup of $2\times$ to $3\times$ in early module tests. This PRNG is integrated across both the communication channel and data source blocks.

Please note that the final numerical outputs will not be completely identical to earlier project phases due to differences in the underlying PRNG stream and seeding sequences. However, the calculated FER/BER trends match class baselines closely (aside from the localized precision losses caused by the quantization block).

To launch the simulation using the Mersenne Twister optimization layer, execute with the flag `--mersenne`.

Function Signatures:

```
void channel_AWGN_add_noise_mersenne(const int32_t *X_N, float *Y_N,
    size_t N, float sigma);
void channel_AWGN_add_noise_i8_mersenne(const int8_t *X_N, float *Y_N,
    size_t N, float sigma);
```

2.7 Channel Optimization

Integrating the vectorized PRNG successfully mitigated the intense computation bottlenecks tied to the channel block. In the standard scalar pipeline, a single call to `rand()` extracts values to compute normalized samples Z_1 (using cosine functions), while throwing away or underutilizing paired Z_2 (using sin) calculations, which wastes valuable processor cycles.

To resolve this, the channel block was redesigned to extract 4 random values per iteration, generating 4 parallel channel output signals simultaneously. This resulted in an immediate throughput increase during initial testing.

Performance was pushed by vectorizing the **Box-Muller** transformation stage while keeping the `sin`, `cos`, and `log` operations scalar. Loop unrolling was then applied to double the workload, processing 8 data elements per loop iteration rather than 4. This corresponds to two parallel random data groups, which are transformed to floating-point metrics via the NEON intrinsic `vcvtq_f32.u32` before normalization.

To optimize further, a precomputed scalar multiplier representing $\frac{1}{\text{UINT32_MAX}}$ was introduced, allowing the use of parallel multiplications instead. While individual noise metrics are still processed sequentially, the resulting noise values are collected into a single vector lane. Scaling by σ and translating the X_N signal format from `int8` to `float` are then handled using parallel SIMD operations.

This implementation doubled the overall throughput of the communication channel. The resulting BER/FER curves match expected classroom profiles closely, with minor variations due to pulling noise values from dual sample pools (Z_1 and Z_2) rather than just one (box-muller provides 2).

3 Throughput Performance Metrics Comparison

This section provides a comparative throughput summary (measured in Mbps) across three test configurations: the **Baseline** pipeline (Lab 3 parameters), the **Mersenne32** layout (NEON-accelerated utilizing a 32-bit Mersenne and partially vectorized Box-Muller function, $K = 32$), and the **Mersenne256** configuration ($K = 256$). Data points are extracted from the finalized convergence SNR points for each simulation run (15 dB for Sim 1, and 12 dB for Sims 2–5).

3.1 Simulation 1 Throughput Comparison (at 15 dB SNR)

Metric	Baseline	Mersenne32	Mersenne256	M32 vs Base	M256 vs Base
Simulation Throughput (Sim_thr)	2.28	8.22	11.42	+261%	+401%
Decoder (Dec_Thr)	61	491	1539	+704%	+2426%
Source (Src_Thr)	27	53	364	+96%	+1255%
Unpacker (Unp_Thr)	0	457	983	-	-
Encoder (Enc_Thr)	53	482	2219	+822%	+4145%
Modulator (Mod_Thr)	1663	1997	8475	+20%	+410%
Channel (Chn_Thr)	12	54	55	+355%	+368%
Demodulator (Dem_Thr)	1671	1708	4224	+2%	+153%
Decoder Stats (Dec_Thr_Stats)	61	491	1539	+704%	+2426%
Monitor (Mon_Thr)	305	527	2920	+73%	+860%
Quantizer (Qua_Thr)	0	461	532	-	-

Table 1: Simulation 1 throughput performance benchmarks at 15 dB SNR.

3.2 Simulation 2 Throughput Comparison (at 12 dB SNR)

Metric	Baseline	Mersenne32	Mersenne256	M32 vs Base	M256 vs Base
Simulation Throughput (Sim_thr)	2.30	8.21	11.48	+257%	+399%
Decoder (Dec_Thr)	162	503	2205	+211%	+1265%
Source (Src_Thr)	29	53	371	+83%	+1201%
Unpacker (Unp_Thr)	0	457	979	-	-
Encoder (Enc_Thr)	53	481	2197	+819%	+4101%
Modulator (Mod_Thr)	1662	1995	8449	+20%	+408%
Channel (Chn_Thr)	12	54	55	+364%	+358%
Demodulator (Dem_Thr)	1678	1748	4239	+4%	+153%
Decoder Stats (Dec_Thr_Stats)	162	503	2205	+211%	+1265%
Monitor (Mon_Thr)	250	528	2906	+112%	+1062%
Quantizer (Qua_Thr)	0	461	549	-	-

Table 2: Simulation 2 throughput performance benchmarks at 12 dB SNR.

3.3 Simulation 3 Throughput Comparison (at 12 dB SNR)

Metric	Baseline	Mersenne32	Mersenne256	M32 vs Base	M256 vs Base
Simulation Throughput (Sim_thr)	2.92	9.85	14.93	+237%	+412%
Decoder (Dec_Thr)	183	510	2483	+179%	+1261%
Source (Src_Thr)	29	53	369	+83%	+1190%
Unpacker (Unp_Thr)	0	457	993	-	-
Encoder (Enc_Thr)	69	494	2520	+623%	+3597%
Modulator (Mod_Thr)	1408	1527	7475	+8%	+431%
Channel (Chn_Thr)	12	53	55	+362%	+379%
Demodulator (Dem_Thr)	1363	1388	3950	+2%	+190%
Decoder Stats (Dec_Thr_Stats)	183	510	2483	+179%	+1261%
Monitor (Mon_Thr)	306	529	2958	+73%	+868%
Quantizer (Qua_Thr)	0	436	528	-	-

Table 3: Simulation 3 throughput performance benchmarks at 12 dB SNR.

3.4 Simulation 4 Throughput Comparison (at 12 dB SNR)

Metric	Baseline	Mersenne32	Mersenne256	M32 vs Base	M256 vs Base
Simulation Throughput (Sim_thr)	4.02	12.28	21.43	+205%	+433%
Decoder (Dec_Thr)	230	512	2777	+123%	+1111%
Source (Src_Thr)	27	52	326	+93%	+1125%
Unpacker (Unp_Thr)	0	456	977	-	-
Encoder (Enc_Thr)	98	507	2901	+417%	+2866%
Modulator (Mod_Thr)	993	1032	5871	+4%	+492%
Channel (Chn_Thr)	12	52	55	+346%	+370%
Demodulator (Dem_Thr)	959	963	3498	+0.4%	+265%
Decoder Stats (Dec_Thr_Stats)	230	512	2777	+123%	+1111%
Monitor (Mon_Thr)	306	527	2945	+72%	+863%
Quantizer (Qua_Thr)	0	394	519	-	-

Table 4: Simulation 4 throughput performance benchmarks at 12 dB SNR.

3.5 Simulation 5 Throughput Comparison (at 12 dB SNR)

Metric	Baseline	Mersenne32	Mersenne256	M32 vs Base	M256 vs Base
Simulation Throughput (Sim_thr)	6.29	16.35	38.05	+160%	+505%
Decoder (Dec_Thr)	262	514	3100	+96%	+1085%
Source (Src_Thr)	27	52	370	+94%	+1282%
Unpacker (Unp_Thr)	0	456	989	-	-
Encoder (Enc_Thr)	175	525	3685	+201%	+2017%
Modulator (Mod_Thr)	516	527	3730	+2%	+623%
Channel (Chn_Thr)	12	50	55	+329%	+373%
Demodulator (Dem_Thr)	505	509	2609	+1%	+417%
Decoder Stats (Dec_Thr_Stats)	262	514	3100	+96%	+1085%
Monitor (Mon_Thr)	288	527	2938	+83%	+920%
Quantizer (Qua_Thr)	0	306	497	-	-

Table 5: Simulation 5 throughput performance benchmarks at 12 dB SNR.

3.6 Summary & Key Findings

3.6.1 Simulation Throughput Performance Summary Table

Simulation	Baseline	Mersenne32 Gain	Mersenne256 Gain
Sim 1 (15 dB)	2.28 Mbps	+261%	+401%
Sim 2 (12 dB)	2.30 Mbps	+257%	+399%
Sim 3 (12 dB)	2.92 Mbps	+237%	+412%
Sim 4 (12 dB)	4.02 Mbps	+205%	+433%
Sim 5 (12 dB)	6.29 Mbps	+160%	+505%
Average	3.76 Mbps	+224%	+430%

Table 6: Consolidated Simulation Speed Improvements Summary.

As demonstrated, the specialized optimization layers applied to our baseline communication chain achieved substantial throughput performance gains at $K = 32$.

Furthermore, data streaming rates scale efficiently with larger frame dimensions ($K = 256$). Attributable to the elimination of loop branch prediction overhead, allowing SIMD vector instructions to maximize hardware occupancy by processing larger continuous blocks of data per execution.

The next section analyzes the relative impacts of these architectural modifications on overall FER and BER curves.

3.6.2 Simulation FER/BER graphs

We noticed minor variations in the FER/BER performance due to the different PRNG stream and its seeding sequence. Additionally, the precision loss from quantization (converting from float to int8) caused some degradation in the curves. Despite these factors, the results remain consistent with the baselines.

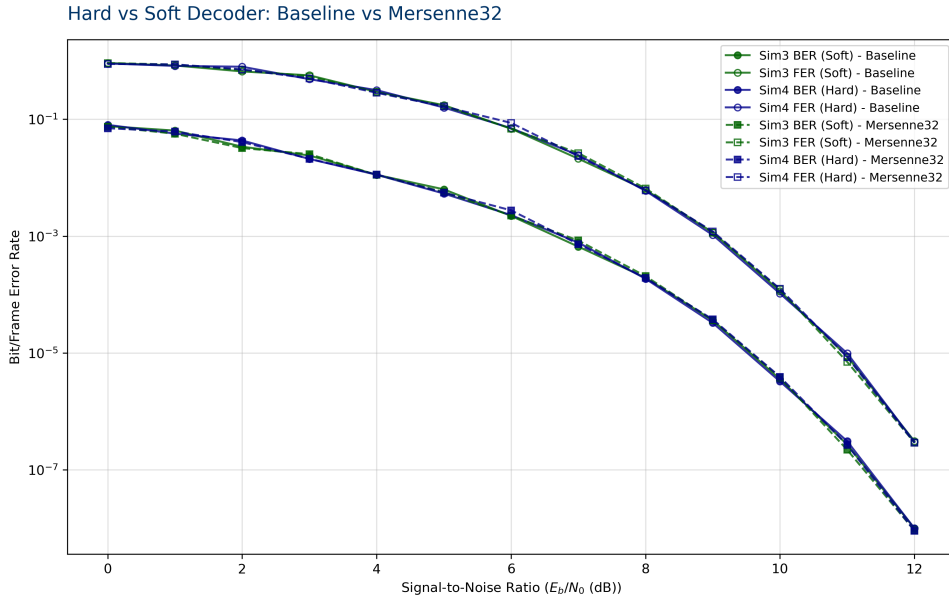


Figure 5: Baseline vs Optimized graphs

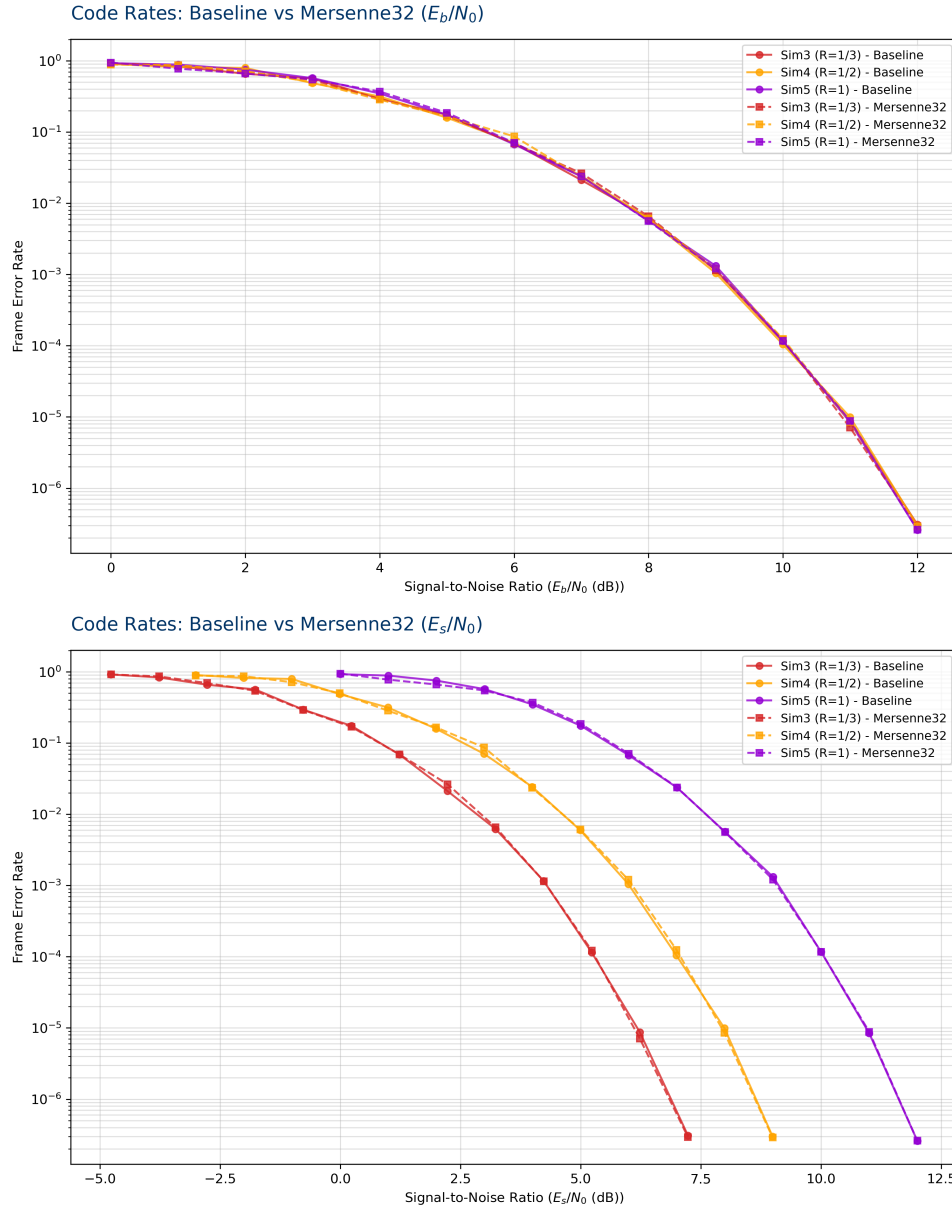
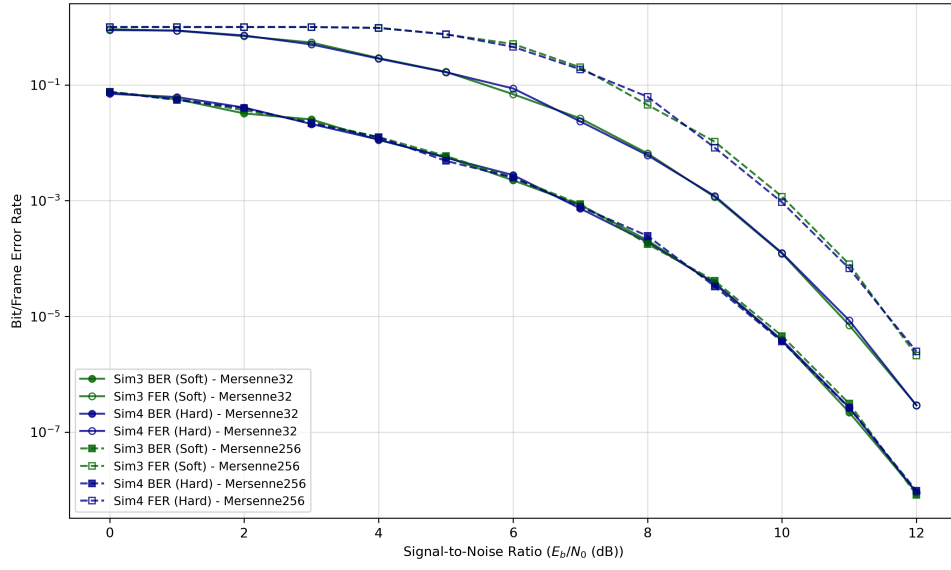


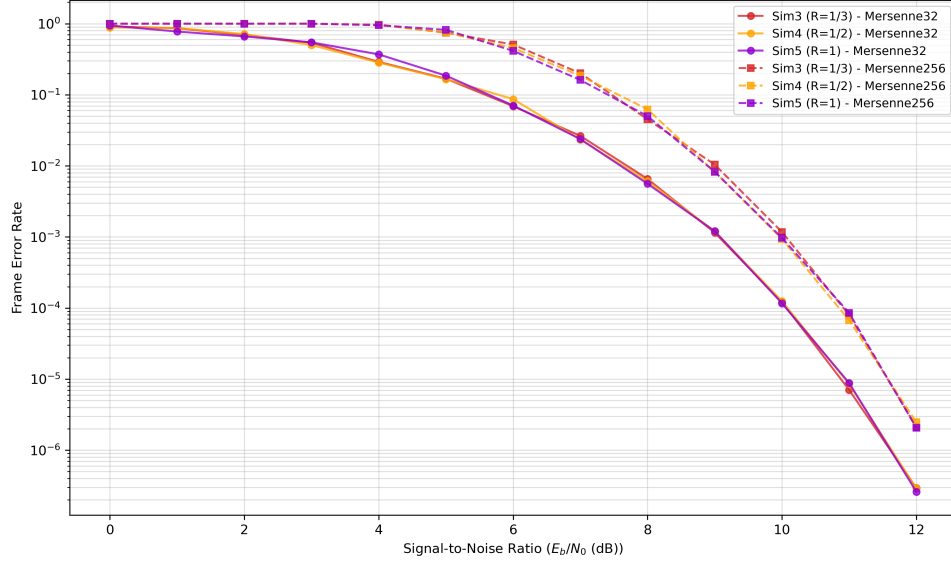
Figure 6: Baseline vs Optimized graphs

The FER is worst with 256 due to the larger frame. More bits = more error, We can achieve better rates if we use different encoding and decoding. Unfortunately that's not the focus here. Graphs down below.

Hard vs Soft Decoder: Mersenne32 vs Mersenne256



Code Rates: Mersenne32 vs Mersenne256 (E_b/N_0)



Code Rates: Mersenne32 vs Mersenne256 (E_s/N_0)

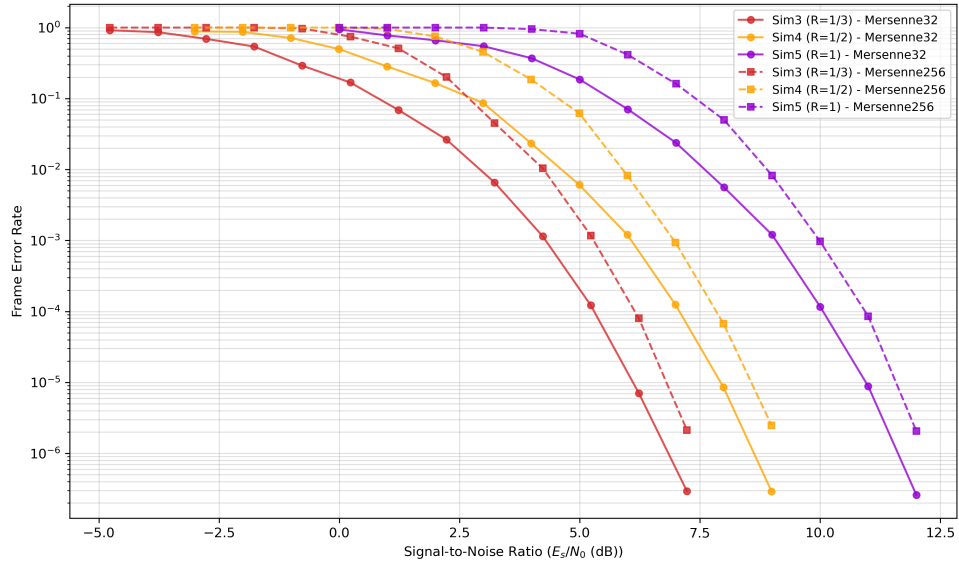


Figure 7: K=32 vs K=256